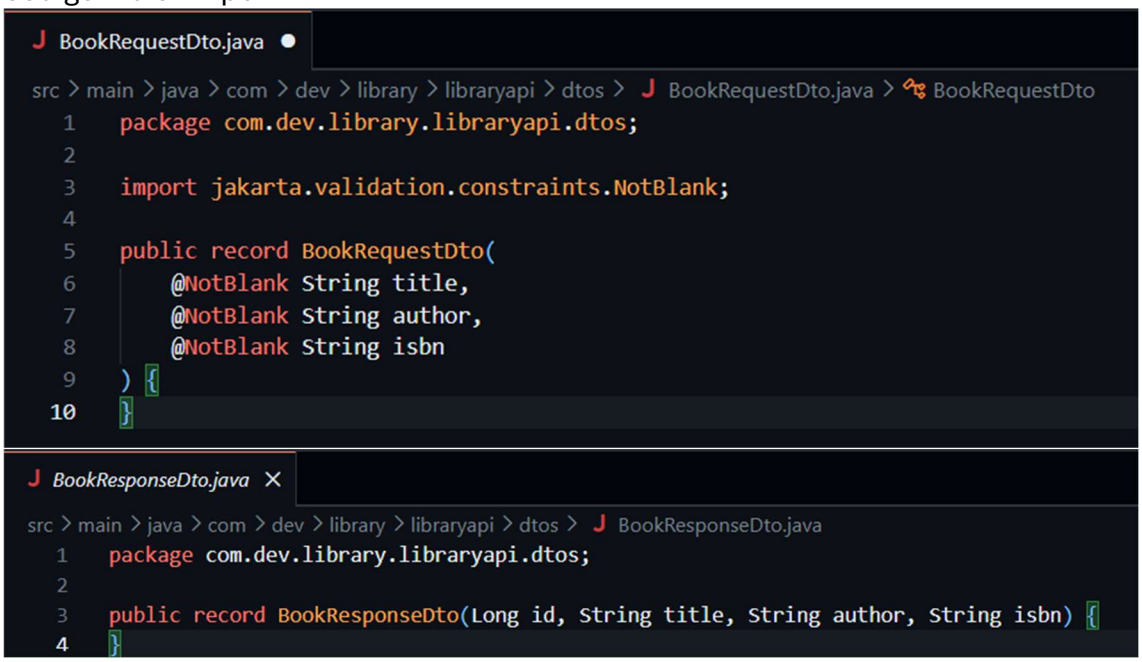
Library API: Ecossistema Seguro para Gestão de Acervos

Tech Stack: Java 17 | Spring Boot 3 | Spring Security | JWT | JUnit 5 | PostgreSQL | Docker

O Problema Real: Em sistemas corporativos, o maior risco não é apenas o armazenamento de dados, mas o acesso não autorizado. Uma API de biblioteca comum permite que qualquer um altere dados. O desafio aqui foi criar uma camada de governança onde apenas usuários autenticados e com papéis específicos possam realizar alterações.

2. Por que Java e Spring Boot? (Escolha técnica)

- **Robustez do Java 17:** Escolhi o Java por sua tipagem forte e maturidade no mercado financeiro e corporativo. O uso de **Records** (introduzidos no Java 17) permitiu criar DTOs solidos, reduzindo bugs de estado e deixando o código mais limpo.



BookRequestDto.java

src > main > java > com > dev > library > libraryapi > dtos > BookRequestDto.java > BookRequestDto

```java
package com.dev.library.libraryapi.dtos;

import jakarta.validation.constraints.NotBlank;

public record BookRequestDto(
    @NotBlank String title,
    @NotBlank String author,
    @NotBlank String isbn
) {
}
```

BookResponseDto.java

src > main > java > com > dev > library > libraryapi > dtos > BookResponseDto.java

```java
package com.dev.library.libraryapi.dtos;

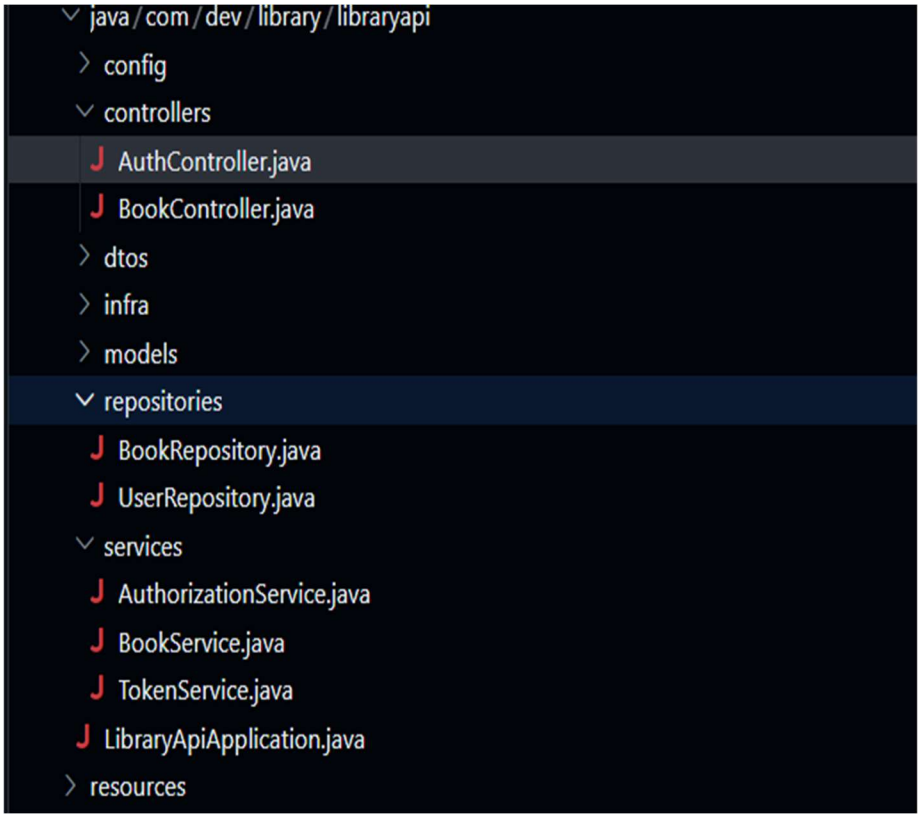
public record BookResponseDto(Long id, String title, String author, String isbn) {
}
```

- **Ecossistema Spring:** O Spring Boot foi utilizado para acelerar o desenvolvimento através da configuração automática. O foco foi usar o **Spring Data JPA** para melhorar a complexidade do SQL, permitindo que eu focasse na regra de negócio e não em comandos de banco de dados.

---

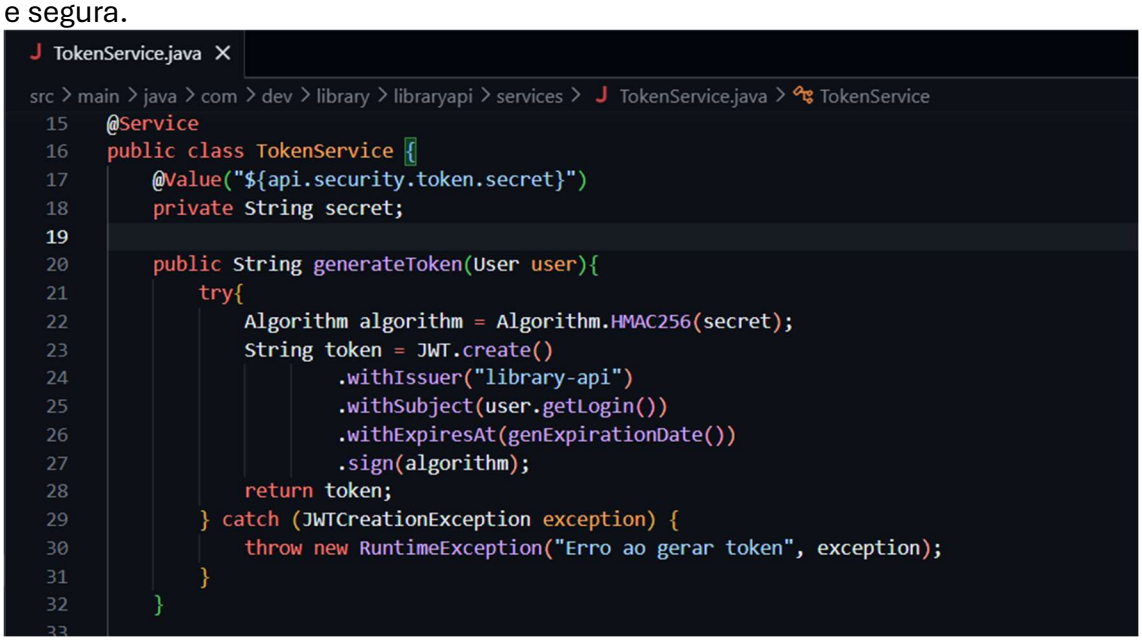
3. Arquitetura:
Dividi o projeto em camadas para resolver o problema da **manutenibilidade**:

- **Controller:** Apenas recebe a requisição. Se a interface mudar, o resto do sistema permanece intacto.
- **Service:** Onde a "mágica" acontece. Aqui estão as regras. Se amanhã a biblioteca mudar a forma de validar um livro, eu só altero um arquivo.
- **Repository:** Isolamento total do banco de dados. Posso trocar o banco sem tocar na lógica de negócio.

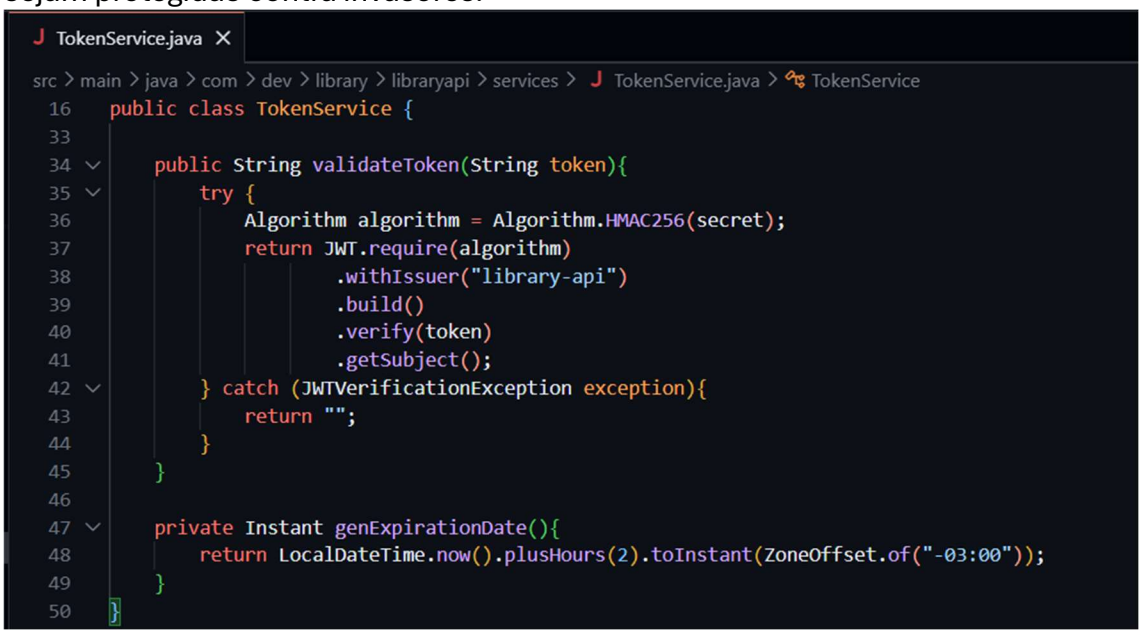


4. Segurança com JWT: O Grande Diferencial

- **O Problema:** Em APIs modernas (REST), manter uma "sessão" aberta no servidor consome muita memória e dificulta a escalabilidade.
- **A Solução (Stateless):** Implementei **JWT (JSON Web Token)**. O servidor não precisa lembrar de ninguém. O usuário apresenta um "crachá digital" assinado. Isso permite que a API atenda milhares de usuários de forma leve e segura.



- **Spring Security:** Configurei filtros customizados que interceptam cada requisição para validar o token, garantindo que rotas de deleção e cadastro sejam protegidas contra invasores.



5. Qualidade: O Seguro contra Erros

- **Testes Unitários (JUnit 5 & Mockito):** Um erro comum em projetos iniciantes é o "medo de mexer no código e quebrar tudo". Resolvi isso criando testes automatizados.
- **Mockito:** Usei para simular o banco de dados. Isso garante que os testes sejam ultra rápidos (milissegundos) e testem apenas a lógica do BookService, sem dependências externas.

```java
@ExtendWith(MockitoExtension.class)
class BookServiceTest {

    @Mock
    private BookRepository bookRepository;

    @InjectMocks
    private BookService bookService;

    @Test
    @DisplayName("Deve salvar um livro com sucesso")
    void saveBookSuccess() {
        // Arrange (Configuração dos dados de teste)
        BookRequestDto dto = new BookRequestDto("O Hobbit", "J.R.R. Tolkien", "123456");
        Book book = new Book();
        book.setId(1L);
        book.setTitle(dto.title());
        book.setAuthor(dto.author());
        book.setIsbn(dto.isbn());

        when(bookRepository.save(any(Book.class))).thenReturn(book);

        // Act (Execução do método a ser testado)
        Book savedBook = bookService.save(dto);

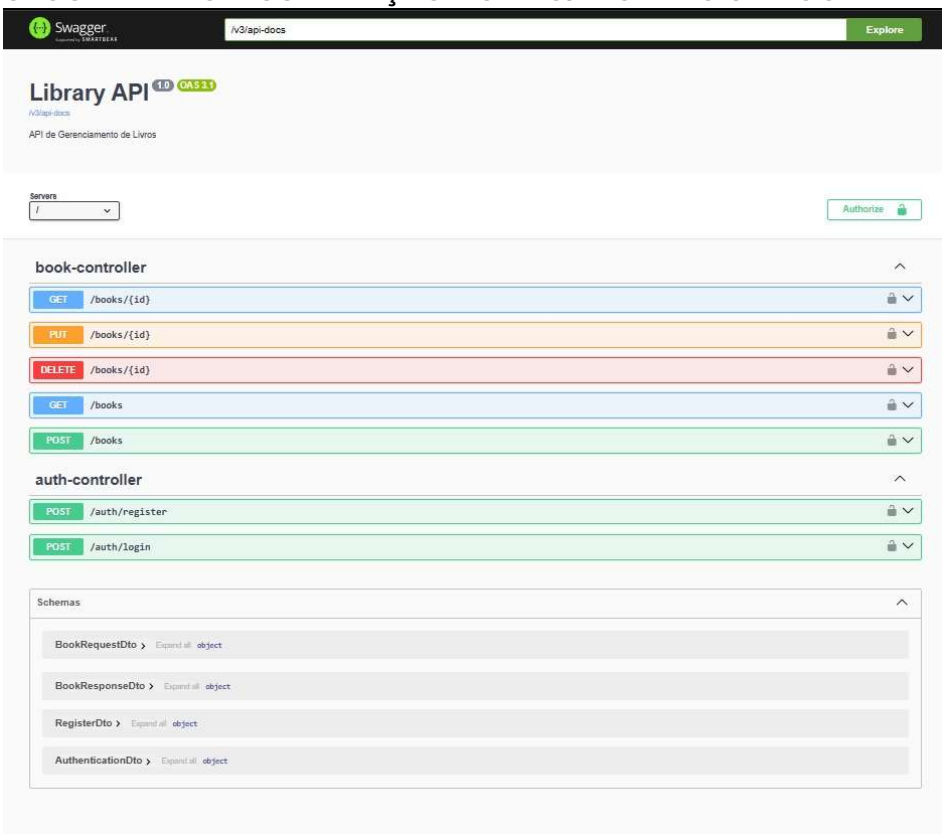
        // Assert (Verificação dos resultados)
        assertNotNull(savedBook);
        assertEquals("O Hobbit", savedBook.getTitle());
        verify(bookRepository, times(1)).save(any(Book.class));
    }
}
```

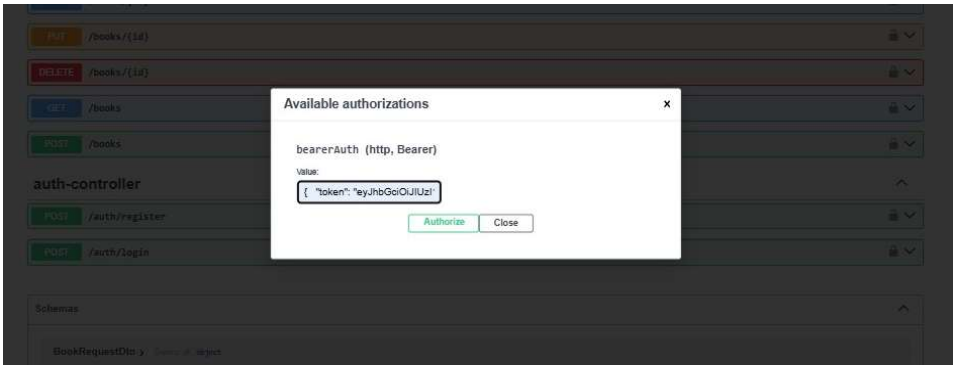


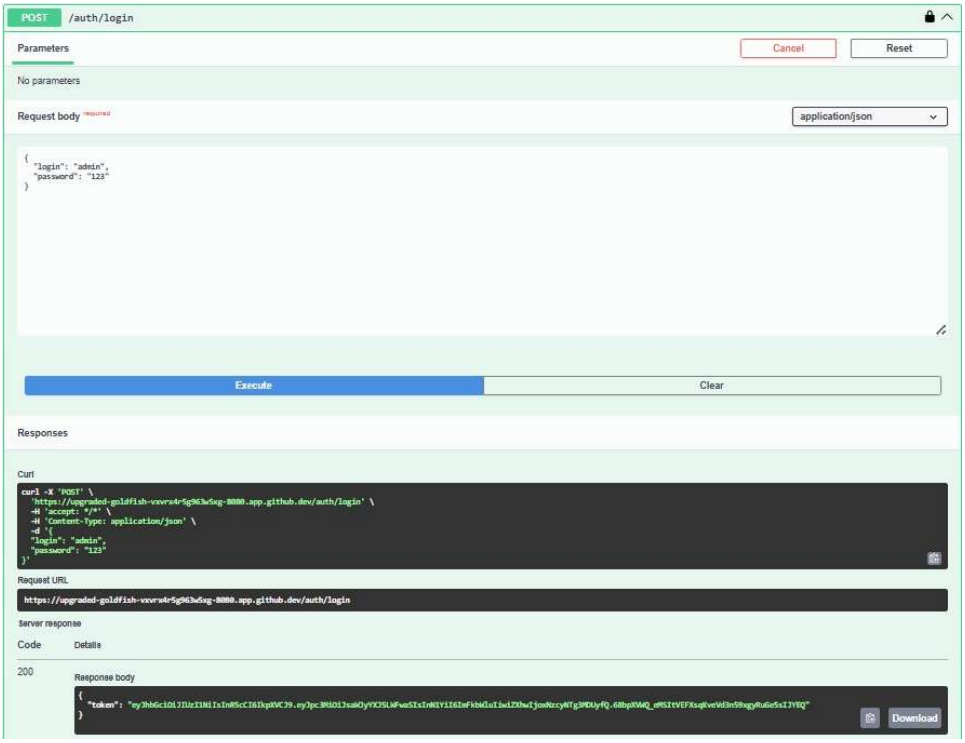
6. Próximos Passos e Visão de Futuro

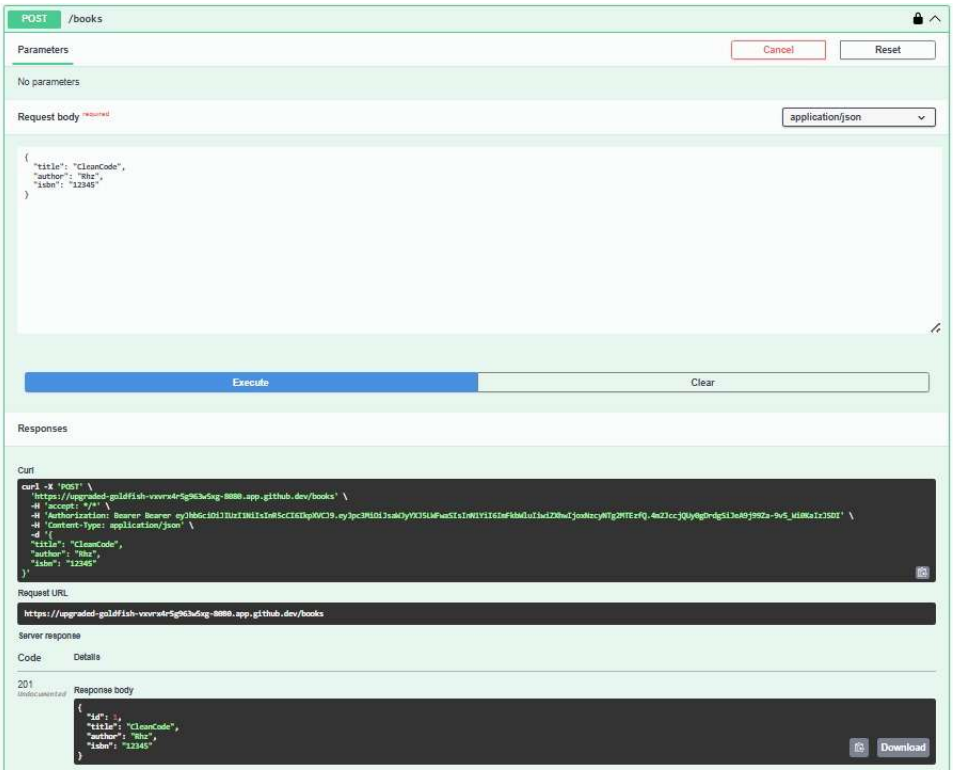
- **Escalabilidade:** O projeto foi pensado para ser **Dockerizado**, facilitando o deploy em nuvem (AWS/Azure).
- **Observabilidade:** Planejo adicionar logs estruturados e métricas para monitorar a saúde da API em tempo real.

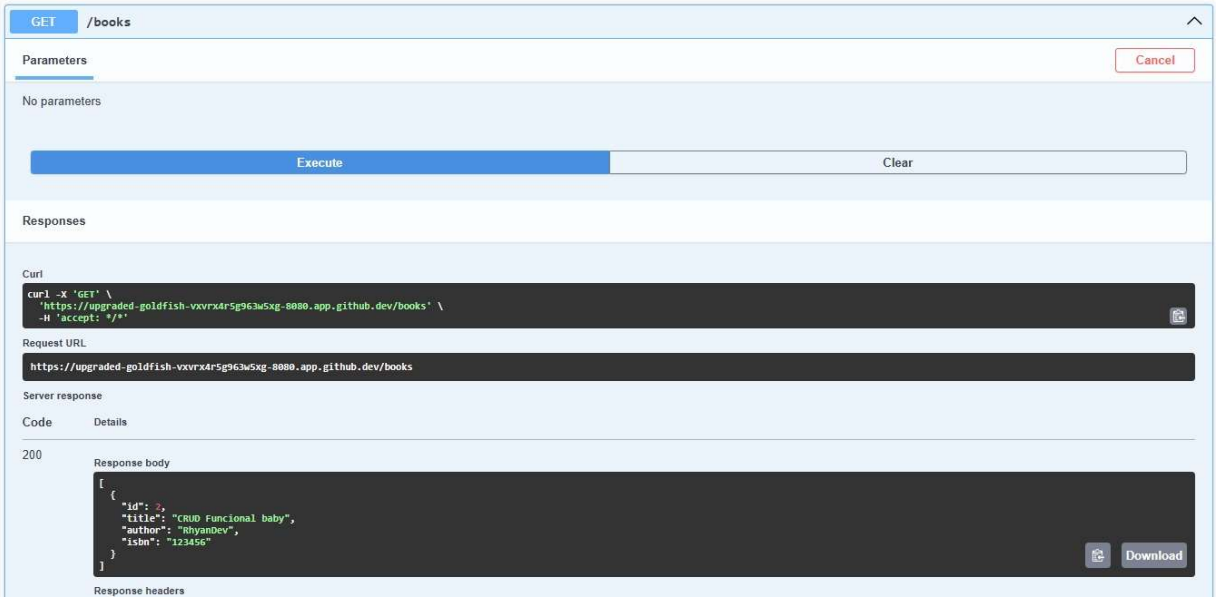
SEGUE ABAIXO VISUALIZAÇÃO DO PROJETO PELO SWAGGER






POST /auth/login
Parameters
Cancel
Reset
No parameters
Request body required
application/json
{
"login": "admin",
"password": "123"
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'https://upgraded-goldfish-vxvrx4r5g963w5xg-8080.app.github.dev/auth/login' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"login": "admin",
"password": "123"
}'
Request URL
https://upgraded-goldfish-vxvrx4r5g963w5xg-8080.app.github.dev/auth/login
Server response
Code
Details
200
Response body
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJsaWQyYXJSLWFwaSIsInN1YiI6ImFkbWluIiwiZXhwIjoxNzcyNTg3MDUyfQ.68bpXVWQ_dMSItVEFXsqKveVd3n59xgyRuGe5sIJYEQ"
}
Download


POST /books
Parameters
Cancel
Reset
No parameters
Request body required
application/json
{
"title": "CleanCode",
"author": "Rhz",
"isbn": "12345"
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'https://upgraded-goldfish-vxvrx4r5g963w5xg-8080.app.github.dev/books' \
-H 'accept: */*' \
-H 'Authorization: Bearer Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJsaWQyYXJSLWFwaSIsInN1YiI6ImFkbWluIiwiZXhwIjoxNzcyNTg2MTErfQ.4mZJccjQUy0gDrdgSi3eA0j99Za-9v5_Wi0KaIzJSDI' \
-H 'Content-Type: application/json' \
-d '{
"title": "CleanCode",
"author": "Rhz",
"isbn": "12345"
}'
Request URL
https://upgraded-goldfish-vxvrx4r5g963w5xg-8080.app.github.dev/books
Server response
Code
Details
201
Undocumented
Response body
{
"id": 1,
"title": "CleanCode",
"author": "Rhz",
"isbn": "12345"
}
Download


GET
/books
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'https://upgraded-goldfish-vxvrx4r5g963w5xg-8080.app.github.dev/books' \
-H 'accept: */*'
Request URL
https://upgraded-goldfish-vxvrx4r5g963w5xg-8080.app.github.dev/books
Server response
Code
Details
200
Response body
[
{
"id": 2,
"title": "CRUD Funcional baby",
"author": "RhyanDev",
"isbn": "123456"
}
]
Download
Response headers